# Homework 3, Machine Learning, Fall 2025

***IMPORTANT* Homework Submission Instructions**

1. All homeworks must be submitted in one PDF file to Gradescope.
2. Please make sure to select the corresponding HW pages on Gradescope for each question.
3. For all coding components, complete the solutions with a Jupyter notebook/Google Colab, and export the notebook (including both code and outputs) into a PDF file. Concatenate the theory solutions PDF file with the coding solutions PDF file into one PDF file which you will submit.
4. You must show work or give an explanation for every question. "Yes" and "No" are not sufficient answers. Always show the steps of your calculations. Only partial credit can be given if you do not explain your answer.
5. Failure to adhere to the above submission format may result in penalties.

As a reminder, don't wait until the last minute to ask for help, because the person you need to speak with might be dealing with many other students close to the deadline. Also, the teaching staff has been instructed that it is not mandatory for them to answer questions on the day the homework is due. So get your questions in early!

You may collaborate on homework, but you must do your own work; we feel this is the best way to learn. If you get stuck, you are welcome to discuss conceptual issues with your classmates or the TAs but you may not copy any code, solutions or proof steps. Do not ask another classmate to perform coding for you. You can look at reference material on the Internet such as ChatGPT, but don't look at that unless you are really stuck, and make sure your answer is complete - don't write "Because I found a theorem that says the answer is X". You must show your complete work for each question. You are required to state what references you used (if you used an outside source). If you've worked with anyone, write down their name below and make sure they've written your name on their homework as well. **Please copy the following statements at the top of your assignment file**:

Agreement 1: This assignment represents my own work. I have worked with the following people and/or LLMs: _____

# 1 Convexity (Zack) - 35 pts

Convexity is a property of functions that makes optimization very nice. In *strictly* convex optimization, every local minimum is also a global minimum. This makes solving for optimal solutions and running gradient descent much easier! In this problem, we'll explore convexity in detail.

Let's start with a few different ways to test if a function is *convex*. A function $f : \mathbb{R} \to \mathbb{R}$ is convex if any one of the following conditions are met (these definitions are not necessarily equivalent):

1. $\forall \lambda \in [0, 1],\ x, y \in \mathbb{R},\ f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$.
   **Any line drawn between two points on the function must lie on or above the function**.
2. $f$ is differentiable and, $\forall x, y \in \mathbb{R}, f(x) \geq f(y) + f'(y)(x - y)$.
   **The graph of a convex function lies on or above all of its tangents**.
3. $f$ is twice differentiable and, $\forall x \in \mathbb{R},\ f''(x) \geq 0$.
   **The graph of a convex function should never curve down**.

Similar definitions apply if $f$ is a function of several variables. The first definition stays the same. In the second definition, the first derivative is replaced with the gradient transpose. In the third definition, the Hessian of $f$ is required to be *positive semidefinite* (which means all eigenvalues are non-negative).

We will now examine various properties of convex functions to better understand some common loss functions. Assume that the domain and codomain of all functions is $\mathbb{R}$ unless stated otherwise. For the following problems, you may prove convexity using any definition above. Note: $f$ is *concave* if $-f$ is convex.

(a) **Additivity.** Let $f$ and $g$ be **convex** functions. Show that $f + g$ is **convex**.

(b) **Log Product.** Let $f_1, \ldots, f_k$ be **concave** functions, $f_i(x) \geq 0 \quad \forall x$. Show that $\ln\left(\prod_{i=1}^{k} f_i\right)$ is **concave**.

(c) **Affine Transformations.** Let $f : \mathbb{R}^m \to \mathbb{R}$ be a **convex** function, and let $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$. Show that $f(Ax + b)$ is **convex**.

(d) **Motivating minimum and maximum.** A line is both convex and concave. Draw four different lines in the Cartesian plane and highlight the minimum and maximum of all four lines for each $x$ in two different colors. These are called the lower and upper envelopes of the lines, respectively. What can you conclude about the lower and upper envelopes of lines in the plane?

(e) **Maximum.** Let $f$ and $g$ be **convex** functions. Show that $\max(f, g)$ is **convex** or give a counterexample.

(f) **Minimum.** Let $f$ and $g$ be **convex** functions. Show that $\min(f, g)$ is **convex** or **concave** or give a counterexample.

(g) **Loss Functions.** We will now examine three common loss functions used for linear models. Let $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ be a dataset, $\boldsymbol{x}_i \in \mathbb{R}^d$, $y_i \in \{-1, 1\}$. Assume that $\boldsymbol{\omega}$ and $b$ are the weights and bias term of a linear model. Show that the following loss functions are **convex** with respect to the weights $\boldsymbol{\omega}$.

   i. **Logistic Loss.**
   $$L(\boldsymbol{\omega}, b) = \sum_{i=1}^{n} \log(1 + e^{-y_i(\boldsymbol{\omega}^T \boldsymbol{x}_i + b)})$$

   ii. **Hinge Loss.**
   $$L(\boldsymbol{\omega}, b, C) = \frac{1}{2}\|\boldsymbol{\omega}\|_2^2 + C \sum_{i=1}^{n} \max(0, 1 - y_i(\boldsymbol{\omega}^T \boldsymbol{x}_i + b)), \quad C \geq 0$$

   iii. **LASSO Regression Loss.**
   $$L(\boldsymbol{\omega}, b, C) = \|\boldsymbol{y} - (X\boldsymbol{\omega} + b\mathbf{1}_n)\|_2^2 + C\|\boldsymbol{\omega}\|_1, \quad C \geq 0$$

## 2 Attention (Theory) (Samarth) - 10 pts

Multi-headed self-attention is the core modeling component of transformers and other advanced architectures known as Large Language Models (LLMs). This question aims to improve our understanding of self-attention equations. We will use the following notation:

- Query vector $\mathbf{q} \in \mathbb{R}^d$, where $d$ is the embedding dimension size hyperparameter.
- Set of value vectors $\mathcal{V} = \{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \ldots, \mathbf{v}_{|\mathcal{D}|}\}$, where $\mathbf{v}_i \in \mathbb{R}^d$ and $\mathcal{D}$ is the set of words in your model's dictionary/vocabulary.
- Set of key vectors $\mathcal{K} = \{\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3, \ldots, \mathbf{k}_{|\mathcal{D}|}\}$, where $\mathbf{k}_i \in \mathbb{R}^d$.

Attention is an operation on query, value, and key vectors. Let's notate attention weights as

$$\alpha_i = \frac{\exp(\mathbf{k}_i^T \mathbf{q})}{\sum_{j=1}^{n} \exp(\mathbf{k}_j^T \mathbf{q})},$$

and let's represent a combined attention embedding as

$$\mathbf{c} = \sum_{i=1}^{n} \mathbf{v}_i \alpha_i.$$

Let's consider a case where the transformer reads a sentence of $n$ words, yielding $n$ value vectors ($\mathbf{v}_1$, $\mathbf{v}_2$, …, $\mathbf{v}_n$) with the corresponding key vectors ($\mathbf{k}_1$, $\mathbf{k}_2$, …, $\mathbf{k}_n$). Let's assume the following:

- The number of key vectors in our sentence is fewer than the number of dimensions ($n < d$)
- All key vectors are **orthogonal** i.e, $\mathbf{k}_i^T \mathbf{k}_j = 0,\ \forall i \neq j$
- All key vectors have **norm 1**, i.e., $\|\mathbf{k}_i\|_2 = 1,\ \forall i \in [1, n]$

In terms of $\mathbf{k}$, report the mathematical expression for a set of possible query vectors $\mathbf{q}$ such that $\mathbf{c}$ is the **average** over all vectors (i.e. $\mathbf{c} \approx \frac{1}{n}(\mathbf{v}_1 + \mathbf{v}_2 + \mathbf{v}_3 + \cdots + \mathbf{v}_n)$). This should be true for any collection of $\mathbf{v}_1, \ldots, \mathbf{v}_n$. Demonstrate all the steps and justify your answer.

## 3 Transformers Implementation for GPTs (Coding) (Muchang) - 20 pts

Generative Pretrained Transformers (GPTs) have recently gained prominence as the state-of-the-art architecture for many NLP tasks. If you've ever used ChatGPT, as the name suggests, you've used a GPT model. In the following question, you will develop a `Pytorch` implementation of the multi-head attention layer of a lightweight GPT model. Using a preprocessed dataset of Shakespeare's plays with a character-level tokenization, you'll then train the model to generate novel Shakespearean text.

Before beginning to code, you will need to set up your environment. First, git clone this repository and create a `conda` environment with `numpy` (`conda install numpy`) and `torch` (`pip install torch`) installed. Make sure to activate it before you start.

Based on our experiments, the model *should* be lightweight enough to be trained on a personal machine. As another option, you may also use your Google Cloud credits to reserve a VM on Google Cloud Platform (GCP) for this question. See the announcement on Ed for more details on obtaining credits. Alternatively, you can also reserve a VM through Duke OIT, though it may be less powerful than a GCP VM.

To briefly go over this repository,

1. `model.py` defines the actual transformer model that we will train. Note that a transformer model consists of multiple *self-attention modules* which each consist of a *self-attention layer* plus a simple *multilayer perceptron* (MLP), including nonlinearities and normalization layers, which help with training. The `Block` class composes both the `SelfAttention` class and `MLP` class for another layer of abstraction.

2. These decoder layer modules are stacked on top of each other within the transformer model, which you can see in the `transformer` attribute of the `Transformer` class.

3. `train.py` is a script that will train the model.

4. `sample.py` is a script that will generate text from the model.

To set up some notation, if the embedding dimension is $C$, and the sequence length is $L$, then a sequence of text inputs can be written as a matrix of shape $L \times C$, also called a *rank 2 tensor*.[1]

To parallelize these operations, PyTorch supports *batching* of inputs, allowing you to input a collection of $B$ matrices of shape $L \times C$, stored in a *rank 3 tensor* of shape $B \times L \times C$, and performing the same operations on every single matrix at once. This also helps with parallelizing matrix multiplication (and other operatons) over higher order tensors. For example, if $A$ has shape $B_1 \times \ldots \times B_k \times L \times M$ and $B$ has shape $B_1 \times \ldots \times B_k \times M \times N$ , then $AB$ has shape $B_1 \times \ldots \times B_k \times L \times N$. We are essentially matrix multiplying over the last two ranks. You should get familiar with this concept of batching, as it will save you a lot of time and effort compared to using for loops. *We do not recommend using for loops, as both the code tends to get messy and the runtime will be significantly slower.*

**(a)** To begin, we will first perform a quick sanity check to ensure your environment has been set up correctly: run the command `python sample.py` from the base of the repository to generate a sample of text. You should notice that the results just look like a random sequence of characters. There are two reasons for this: we haven't implemented the attention layer, nor have we trained the model.

**(b)** Let's focus on the attention layer first. We will do this step by step by implementing the `forward` method in `SelfAttention` class in `model.py`. Recall that the formula for scaled-dot product single-headed attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{C}}\right)V \tag{1}$$

where $Q, K, V$ are the query, key, value matrices, and the softmax operation is done over the *rows* of the input matrix. Each of these steps should not take more than 10 lines of code each. We have provided an outline in the codebase as a guide, but you do not necessarily have to follow it.

i. The query, key, and value matrices are computed by `self.c_attn`, which is a linear layer $\ell$ that maps each token $\mathbf{x}$ to its associated key, query, and value vectors through the map

$$\ell(\mathbf{x}; \mathbf{A}, \mathbf{b}) = \mathbf{A}\mathbf{x} + \mathbf{b} = \begin{pmatrix} \mathbf{q} \\ \mathbf{k} \\ \mathbf{v} \end{pmatrix} \tag{2}$$

where $\mathbf{x} \in \mathbb{R}^C$, $\mathbf{A} \in \mathbb{R}^{3C \times C}$, and $\mathbf{b} \in \mathbb{R}^{3C}$. This is batched over the entire sequence length $L$ and over the $B$ batches for each sequence, so our output wouldn't be simply in $\mathbb{R}^{3C}$, but rather of size $B \times L \times 3C$. You want to take this output matrix of parameters and partition it into the query, key, value matrices, each of size $B \times L \times C$.

[1] A scalar is a rank 0 tensor. A vector, which is an array of scalars, is a rank 1 tensor. A matrix, which is an array of vectors, is a rank 2 tensors. So on and so forth.

ii. We want to implement *multihead* attention by further partitioning each matrix into $H$ submatrices, where $H$ is the number of heads. $H$ is available as `self.n_head` within the class.

$$Q \mapsto (Q_1, \ldots, Q_H)$$
$$K \mapsto (K_1, \ldots, K_H)$$
$$V \mapsto (V_1, \ldots, V_H)$$

To implement multihead attention, we want to add an extra rank to change the shape of each matrix from $(B \times L \times C) \mapsto (B \times H \times L \times C/H)$. Note that $H$ must evenly divide $C$ since we want to evenly partition the embedding dimension, so $C/H$ will always be an integer. The same logic should be looped through the $K$ and $V$ matrices as well. A visual is provided for some intuition.


`2024 Examples/multihead_visual.png`

*Figure 1:* A visualization of how a single element of a batch, of shape $L \times C$, should be reshaped to shape $H \times L \times C/H$. You want to add an extra dimension by stacking these submatrices on top of each other. In the end, we should satisfy `Q[i, j, :, :].shape = (T, C // H)`.

iii. Now we must actually implement the attention operation described in the equation above. First, implement only the operation

$$\frac{Q_i K_i^T}{\sqrt{C/H}} \tag{3}$$

for all $i = 1, \ldots H$ over all $B$ batches (note that there is no index over batches notated, which is perhaps a fault of the notation, but this is common in transformer papers to leave out an index). Once this is done, your result should be of shape $B \times H \times L \times L$.

iv. We also need a masking step since this is a decoder architecture. We have provided a `self.mask` attribute consisting of a lower triangular matrix of 1s. Use this to mask your output so that future tokens are ignored in the attention score. You might find the `torch.Tensor.masked_fill()` operation helpful.

v. Then, we can apply the softmax operation on the rows of each $L \times L$ matrix within each head and batch. After this should be a dropout layer, which we have implemented for you. Finally, we can multiply by the value matrix $V$ to get the head outputs $y$. After this step, the assertion statement that we have implemented on `y.shape` should hold true.

**(c)** Train this model using the command `python train.py` and plot the loss curve over time. Note that training for 2000 epochs on Google Cloud is estimated to take between 5-20 minutes. You should allocate your time accordingly for training.

**(d)** Generate a new brief Shakespearean text using the command `python sample.py` and post your output. Now you have learned the details of a minimal transformer architecture!

# 4 Attention and Convolution (Coding) (James) - 35 pts

In this question, we will use the PyTorch framework to explore some properties of self-attention and some connections between self-attention and convolution[2]. First, we define the notion of a "similarity metric", which can be thought of as the opposite of a distance metric. Formally, we call a function $s : \mathbb{R}^{d \times d} \to \mathbb{R}$ a similarity metric if, for any $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{R}^d$:

1. $s$ is bounded above by some value $b$: $s(\mathbf{x}, \mathbf{y}) \leq b$.

[2]In this problem "convolution" refers to the operation used in convolutional neural networks. Formally, we are actually going to discuss cross-correlation, but the machine learning community refers to this operation as convolution by convention.

2. The similarity between an object and itself is the maximum similarity possible: $s(\mathbf{x}, \mathbf{x}) = b$
3. The similarity between two distinct objects is not the maximum similarity: $\mathbf{x} \neq \mathbf{y} \implies s(\mathbf{x}, \mathbf{y}) < b$
4. Similarity is symmetric: $s(\mathbf{x}, \mathbf{y}) = s(\mathbf{y}, \mathbf{x})$

**(a)** When learning about convolution, you may have learned that it is similar to template matching, where we find the similarity between a template and an image at each location. In the first part of this problem, we will use the PyTorch package to visually explore whether convolution is computing a similarity function. First, recall the definition of convolution. For an input matrix $\mathbf{Z} \in \mathbb{R}^{d_{\text{in}} \times h_{\text{in}} \times w_{\text{in}}}$ and a learned tensor of $d_{\text{out}}$ convolutional filters $\mathbf{K} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}} \times h_{\text{kernel}} \times w_{\text{kernel}}}$, the output of a simple 2 dimensional convolution at position $h, w$ for filter $j$ is:

$$(\mathbf{Z} * \mathbf{K})_{j,h,w} = \sum_{a=1}^{d_{\text{in}}} \sum_{b=1}^{h_{\text{kernel}}} \sum_{c=1}^{w_{\text{kernel}}} k_{j,a,b,c} z_{a,h+b-\lfloor h_{\text{kernel}/2} \rfloor, w+c-\lfloor h_{\text{kernel}/2} \rfloor}, \tag{4}$$

where the $*$ operator denotes convolving the first tensor with the second. For simplicity, in this problem we will consider convolution with $h_{\text{kernel}} = w_{\text{kernel}} = 1$, simplifying Equation **??** to

$$(\mathbf{Z} * \mathbf{K})_{j,h,w} = \sum_{a=1}^{d_{\text{in}}} k_{j,a,1,1} z_{a,h,w}. \tag{5}$$

We will also consider $d_{\text{in}} = 3$ to be the RGB representation of a pixel at each location, allowing easy visualization. The starting notebook available at this link provides an input tensor and a kernel tensor. Use this starting notebook to:

i. Visualize the provided input tensor using Matplotlib's imshow function. Additionally, visualize each filter in the given "kernels" tensor as a separate subplot in one figure. Note that PyTorch and Matplotlib follow different conventions around the semantics of each axis in a tensor. In PyTorch, an image is represented with the channel dimension (e.g., RGB) first, followed by height and width. In Matplotlib, the convention is height and width followed by channel. To display torch tensors using matplotlib, you will need to reorder the axes using torch.permute such that the channel dimension comes last rather than first. Note that the color of each filter appears in the input tensor.

ii. Use the PyTorch function torch.nn.functional.conv2d to convolve the given input tensor with the given kernels. This should produce an output tensor in $\mathbb{R}^{d_{\text{out}} \times h_{\text{in}} \times w_{\text{in}}}$. Display the activation map for each kernel as a subplot of one plot, i.e., produce $d_{\text{out}}$ subplots of shape $h_{\text{in}} \times w_{\text{in}}$. Based on these visualizations, you should notice that the entries in these matrices are not produced by a similarity function. State which of the rules for similarity functions is violated and how you know.

iii. Let's take a moment to consider something that *is* a similarity function. For two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, the cosine similarity between $\mathbf{x}$ and $\mathbf{y}$ is defined to be:

$$\text{CosSim}(\mathbf{x}, \mathbf{y}) := \cos(\theta(\mathbf{x}, \mathbf{y})), \tag{6}$$

where $\theta : \mathbb{R}^{d \times d} \to [0, 2\pi)$ is a function that gets the angle between two vectors. Prove that CosSim is a similarity metric. Hint: $\theta$ is a distance metric.

iv. Implement CosSim using a processing step on $\mathbf{Z}$ and $\mathbf{K}$ and torch.nn.functional.conv2d; that is, perform a transformation $T$ on $\mathbf{Z}$ and $\mathbf{K}$ such that $(\bar{\mathbf{Z}} * \bar{\mathbf{K}})_{j,h,w}$ computes a cosine similarity, where $\bar{\mathbf{Z}} = T(\mathbf{Z})$ and $\bar{\mathbf{K}} = T(\mathbf{K})$. Display the activation map for each kernel. Do the activation maps reflect a similarity function? What is the difference between what you just implemented and a normal convolution? Hint: Recall that $\mathbf{x}^T \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos(\theta(\mathbf{x}, \mathbf{y}))$.

**(b)** We now attend to attention. Recall the definition of dot product attention as computed in "Attention is All You Need". For matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{w_{\text{in}} \times d_{\text{in}}}$, we have

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) := \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_{\text{in}}}}\right)\mathbf{V}. \tag{7}$$

In the simplest form of self-attention, an input matrix $\mathbf{Z} \in \mathbb{R}^{w_{\text{in}} \times d_{\text{in}}}$ is passed as all three arguments, i.e.

$$\text{SimpleSelfAttention}(\mathbf{Z}) := \text{Attention}(\mathbf{Z}, \mathbf{Z}, \mathbf{Z}) = \text{softmax}\left(\frac{\mathbf{Z}\mathbf{Z}^T}{\sqrt{d_{\text{in}}}}\right)\mathbf{Z}.$$

1. Note that you were given an input image in $\mathbb{R}^{d_{in} \times h_{in} \times w_{in}}$, which does not quite line up with the dimensions expected for attention. Flatten the given input into a matrix of shape $(d_{in},\ h_{in}w_{in})$, transpose the resulting matrix to shape $(h_{in}w_{in},\ d_{in})$, and use it to compute $\mathbf{Z}\mathbf{Z}^T$. Visualize the resulting matrix with imshow. Based on this visualization, could the entries in the resulting matrix be produced by a similarity function? State whether one of the rules for similarity functions is violated and how you know.

2. Compute softmax $\left(\frac{\mathbf{Z}\mathbf{Z}^T}{\sqrt{d_{\text{in}}}}\right)$, and visualize the resulting matrix. Based on this visualization, could the entries in this matrix be produced by a similarity function? State whether a rule for similarity metrics is violated and how you know.

3. Let $\bar{\mathbf{Z}} := \begin{bmatrix} 1/\|\mathbf{z}_{1,:}\|_2 & 0 & \dots & 0 \\ 0 & 1/\|\mathbf{z}_{2,:}\|_2 & \dots & 0 \\ \vdots & & & \vdots \\ 0 & 0 & \dots & 1/\|\mathbf{z}_{h_{in} \times w_{in},:}\|_2 \end{bmatrix}$. So the resulting matrix $\bar{\mathbf{Z}}$ should be in the shape of $(h_{in} \times w_{in}, h_{in} \times w_{in})$. Finally, compute softmax $\left(\frac{\bar{\mathbf{Z}}(\mathbf{Z}\mathbf{Z}^T)\bar{\mathbf{Z}}}{\sqrt{d_{\text{in}}}}\right)$, and visualize the resulting matrix (Note: The softmax should be applied across the resulting matrix, so you should flatten it before applying the softmax operation in PyTorch and reshape it after). Based on this visualization, could the entries in this matrix be produced by a similarity function? State whether one of the rules for similarity functions is violated and how you know.

4. Based on the prior questions, you can see a connection between attention and convolution. Implement the function $f(\mathbf{Z}) := \frac{\bar{\mathbf{Z}}(\mathbf{Z}\mathbf{Z}^T)\bar{\mathbf{Z}}}{\sqrt{d_{\text{in}}}}$ using a convolution; reshape the output to a matrix of the appropriate size, and visualize the result. Hint: Recall that $\frac{\bar{\mathbf{Z}}(\mathbf{Z}\mathbf{Z}^T)\bar{\mathbf{Z}}}{\sqrt{d_{\text{in}}}} \in \mathbb{R}^{w_{\text{in}}h_{\text{in}} \times w_{\text{in}}h_{\text{in}}}$ and $(\mathbf{Z} * \mathbf{K}) \in \mathbb{R}^{d_{\text{out}} \times h_{\text{in}} \times w_{\text{in}}}$. How many convolutional kernels will you need to make these dimensions match?